

PYTHON PROGRAMLAMA DÖNEM ÖDEVİ

Konu: Akıllı Müşteri Yönetim ve Analiz Sistemi (Telco Senaryosu) **Ortam:** Google Colab

Workflows:

Kısım 1: Veri Yapıları ve Temel Mantık

WF1) Değişkenler ve Tipler: Bir müşterinin `ad_soyad` (string), `aylik_ücret` (float), `sadakat_ayi` (int) ve `aktif_mi` (boolean) bilgilerini içeren değişkenler tanımlandı.

```
1 # 1. Metin (String) tipi: Çift tırnak içinde yazılır.  
2 ad_soyad = "Tarık Doğan"  
3  
4 # 2. Ondalıklı Sayı (Float) tipi: Virgül yerine nokta (.) kullanılır.  
5 aylik_ucret = 549.90  
6  
7 # 3. Tam Sayı (Integer) tipi: Direkt sayı olarak yazılır.  
8 sadakat_ayi = 28  
9  
10 # 4. Mantıksal (Boolean) tip: Sadece True (Doğru) veya False (Yanlış) değerini alır.  
11 aktif_mi = True
```

WF2) Liste ve Sözlük Kullanımı: Şirketin sunduğu 5 farklı hizmeti bir Listeye, müşterinin tüm bilgilerini ise bir Sözlüğe (Dictionary) kaydedildi.

```
1 # 5 farklı hizmeti içeren bir Liste  
2 hizmetler = ["İnternet", "Televizyon", "Mobil Veri", "Sabit Telefon", "Bulut Depolama"]
```

```
1 # Müşteri bilgilerini içeren bir Sözlük
2 muster_i_sozlugu = {
3     "ad_ soyad": ad_ soyad,
4     "aylik_ ucret": aylik_ ucret,
5     "sada kat_ ayi": sada kat_ ayi,
6     "aktif_ mi": aktif_ mi
7 }
8 # Az önce tanımlanan değişkenler buraya çekildi.
```

Sözlüğün mantığı "Anahtar" : "Değer" (Key : Value) şeklindedir.

WF3) Koşullu İfadeler (If-Else): Müşterinin aylık_ücret tutarı 500 TL'den fazlaysa veya sada kat_ ayi 24 aydan uzunsa ekrana "VIP Müşteri: İndirim Tanımla" yazdıran, aksi halde "Standart Müşteri" yazdıran kodu yazdırıldı.

```
1 # Müşteri sözlüğündeki verileri kullanarak karar veriyoruz
2 if muster_i_sozlugu["aylik_ ucret"] > 500 or muster_i_sozlugu["sada kat_ ayi"] > 24:
3     print("VIP Müşteri: İndirim Tanımla")
4 else:
5     print("Standart Müşteri")
6
7
```

VIP Müşteri: İndirim Tanımla

WF4) String Operasyonları: Müşterinin adını tamamen büyük harfe çevrildi ve customerID formatında rastgele bir ID oluşturuldu.

```
1 import random
2
3 # 1. İsmi Büyük Harfe Çevirme
4 # .upper() fonksiyonu string içindeki tüm karakterleri BÜYÜK harf yapar.
5 ad_ buyuk = muster_i_sozlugu["ad_ soyad"].upper()
6
7 # 2. Rastgele Bir Sayı Üretme
8 # 1000 ile 9999 arasında rastgele bir tam sayı seçiyoruz.
9 rastgele_ sayi = random.randint(1000, 9999)
10
11 # 3. CustomerID Oluşturma (f-string yöntemiyle)
12 # 'f' harfi tırnak içindeki süslü parantezlerin { } içine değişken koymamızı sağlar.
13 customerID = f"IST-2026-{rastgele_ sayi}"
14
15 # Sonuçları görelim
16 print(f"Düzenlenmiş İsim: {ad_ buyuk}")
17 print(f"Oluşturulan Müşteri ID: {customerID}")
```

Düzenlenmiş İsim: TARIK DOĞAN
Oluşturulan Müşteri ID: IST-2026-9704

Teslim Edilecekler:

- Yazdığınız kodun Colab linki veya PDF çıktısı.
- Kritik Soru Cevabı: "Müşteri bilgilerini neden bir 'Liste' yerine 'Sözlük' içinde saklamayı tercih ettik? Avantajı nedir?"

Okunabilirlik ve Anlamlılık: Sözlükler "Anahtar-Değer" (Key-Value) yapısıyla çalışır. Bu sayede her verinin ne anlama geldiği (örneğin: `sadakat_ay`) kod içerisinde açıkça görülür.

Erişim Kolaylığı: Listede bir veriyi bulmak için sırasını bilmek gerekirken, sözlükte etiketiyle (ID, isim vb.) direkt çağrılabilir.

Hata Payının Azalması: Listenin sırası karıştığında (örneğin ismin önüne yanlışlıkla başka bir veri eklendiğinde) tüm kod bozulur. Ancak sözlükte sıranın bir önemi yoktur; etiket doğru olduğu sürece doğru veriye ulaşırsınız.

Veri Güncelleme: Sözlükte bir bilgiyi güncellemek (örneğin sadece `aylik_ucret` bilgisini değiştirmek) çok daha güvenli ve kolaydır.

Özetle: Liste bir "sıralı yığındır", Sözlük ise "etiketli bir klasör sistemidir". Müşteri gibi karmaşık bilgileri yönetmek için klasör sistemi (sözlük) çok daha profesyonel bir yaklaşımdır.

Kısım 2: Fonksiyonlar, Döngüler ve Kütüphaneler

Amaç: Birden fazla müşteriyi yöneten ve analiz yapan bir sistem kurmak.

Workflows:

WF1) **Döngüler ve Listeler:** İçinde 5 farklı müşterinin sözlük (dictionary) verilerinin bulunduğu bir ana liste (`müşteriler_listesi`) oluşturuldu.

```

1 # 5 farklı müşteri için sözlüklerden oluşan bir liste oluşturuyoruz
2 musteriler_listesi = [
3     {
4         "ad_soyad": "Fatih Eroğlu",
5         "aylik_ucret": 550.0,
6         "sadakat_ay": 30,
7         "aktif_mi": True
8     },
9     {
10        "ad_soyad": "Simge Polat",
11        "aylik_ucret": 420.0,
12        "sadakat_ay": 12,
13        "aktif_mi": True
14    },
15    {
16        "ad_soyad": "Cüneyt Çakır",
17        "aylik_ucret": 600.0,
18        "sadakat_ay": 48,
19        "aktif_mi": False
20    },
21    {
22        "ad_soyad": "Seren Serengil",
23        "aylik_ucret": 350.0,
24        "sadakat_ay": 5,
25        "aktif_mi": True
26    },
27    {
28        "ad_soyad": "Selahattin Çakaler",
29        "aylik_ucret": 750.0,
30        "sadakat_ay": 25,
31        "aktif_mi": True
32    }
33 ]
34
35 # Listenin başarıyla oluşturulduğunu kontrol etmek için uzunluğunu yazdıralım
36 print(f"Toplam müşteri sayısı: {len(musteriler_listesi)}")

```

Toplam müşteri sayısı: 5

WF2) Fonksiyon Tanımlama: Bir fonksiyon yazıldı (tutar_hesapla). Bu fonksiyon parametre olarak aylık_ucret alıyor ve üzerine %20 KDV ekleyip geri döndürüyor (return).

```

1 # Fonksiyon
2 def tutar_hesapla(aylik_ucret):
3     """
4     Parametre olarak gelen ücrete %20 KDV ekler.
5     """
6     kdv_dahil_tutar = aylik_ucret * 1.20
7     # %20 eklemek demek 1.20 ile çarpmak demektir
8     return kdv_dahil_tutar
9
10 # Fonksiyon testi (Örn: 100 TL için sonuç 120.0 olmalı)
11 test_tutari = 100
12 sonuc = tutar_hesapla(test_tutari)
13 print(f"Test Sonucu: 100 TL -> {sonuc} TL (KDV Dahil)")

```

Test Sonucu: 100 TL -> 120.0 TL (KDV Dahil)

WF3) **Kütüphane Kullanımı:** math kütüphanesini kullanarak toplam fatura tutarını yukarıya yuvarlandı. datetime kütüphanesi ile bugünün tarihini faturanın altına eklendi.

```
1 import math      # Yuvarlama işlemleri için
2 import datetime  # Tarih bilgisi için
3
4 # Örnek bir müşteri ücreti alalım (Önceki listeden veya rastgele)
5 orijinal_ucret = 550.45
6
7 # 1. Daha önce yazdığımız fonksiyonu kullanarak KDV'li tutarı hesaplayalım
8 kdv_li_tutar = tutar_hesapla(orijinal_ucret) # Sonuç örn: 660.54
9
10 # 2. math kütüphanesi ile YUKARI yuvarlama
11 # math.ceil() fonksiyonu sayıyı ne olursa olsun bir üst tam sayıya tamamlar.
12 toplam_fatura = math.ceil(kdv_li_tutar)
13
14 # 3. datetime kütüphanesi ile bugünün tarihini alma
15 bugun = datetime.datetime.now()
16 tarih_formati = bugun.strftime("%d/%m/%Y") # Gün/Ay/Yıl formatına çevirdik
17
18 # Sonuçları Yazdıralım
19 print(f"KDV Dahil Ham Tutar: {kdv_li_tutar:.2f} TL")
20 print(f"Yuvarlanmış Toplam Fatura: {toplam_fatura} TL")
21 print("-" * 30)
22 print(f"Fatura Tarihi: {tarih_formati}")
```

KDV Dahil Ham Tutar: 660.54 TL
Yuvarlanmış Toplam Fatura: 661 TL

Fatura Tarihi: 13/05/2026

WF4) **Hata Denetimi ve Set Kullanımı:** Şirketin verdiği tüm hizmetleri (tekrarlı olabilir) bir listeye atıldı, ardından set() fonksiyonunu kullanarak **benzersiz** hizmet listesi ekrana bastırıldı.

```
1 # 1. Tekrarlı hizmetlerin olduğu liste (Bazı hizmetler bilerek birden fazla yazıldı)
2 tum_verilen_hizmetler = [
3     "İnternet", "Televizyon", "İnternet",
4     "Mobil Veri", "Televizyon", "Bulut Depolama",
5     "İnternet", "Sabit Telefon", "Mobil Veri"
6 ]
7
8 # 2. set() fonksiyonu ile benzersiz olanları bulma
9 # set yapısı, matematiksel kümeler gibidir;
10 # içinde bir elemandan sadece bir tane barındırır.
11 benzersiz_hizmetler = set(tum_verilen_hizmetler)
12
13 # 3. Sonuçları ekrana basma
14 print("Tüm Hizmet Kayıtları (Tekrarlı):", tum_verilen_hizmetler)
15 print("-" * 30)
16 print("Benzersiz Hizmet Listesi:", benzersiz_hizmetler)
17
18 # tekrar listeye çevirme
19 benzersiz_liste = list(benersiz_hizmetler)
```

Tüm Hizmet Kayıtları (Tekrarlı): ['İnternet', 'Televizyon', 'İnternet', 'Mobil Veri', 'Televizyon', 'Bulut Depolama', 'İnternet', 'Sabit Telefon', 'Mobil Veri']

Benzersiz Hizmet Listesi: {'Mobil Veri', 'Televizyon', 'Bulut Depolama', 'Sabit Telefon', 'İnternet'}

WF5) **AI Denetimi (Önemli):** Yapay zekaya bu ödevi yaptırdı. AI'nın yazdığı kodda hata varsa düzeltildi. Kodun her satırına # ile o satırın hangi konuya (Örn: #Döngü Kullanımı) ait olduğunu yazıldı.

Python

```
import datetime # Kütüphane Kullanımı (Zaman ve Tarih)

# --- 1. KISIM: TEMEL MANTIK ---

# Değişken Tanımlama ve Veri Tipleri (String, Float, Int, Boolean)
ad_soyad = "Ahmet Yılmaz"
aylik_ucret = 550.0
sadakat_ayi = 30
aktif_mi = True

# Liste Kullanımı (Veri Yapıları)
hizmetler = ["İnternet", "Televizyon", "Mobil Veri", "Sabit Telefon", "Bulut Depo"]

# Sözlük (Dictionary) Kullanımı (Veri Yapıları)
musteri_verisi = {
    "ad_soyad": ad_soyad,
    "aylik_ucret": aylik_ucret,
    "sadakat_ayi": sadakat_ayi,
    "aktif_mi": aktif_mi
}

# Koşullu İfadeler ve Mantıksal Operatörler (If-Else / Or)
if muster_i_verisi["aylik_ucret"] > 500 or muster_i_verisi["sadakat_ayi"] > 24:
    status = "VIP Müşteri: İndirim Tanımla" # Koşul sağlanırsa çalışır
else:
    status = "Standart Müşteri" # Koşul sağlanmazsa çalışır
```

```

# String Metotları (Büyük Harfe Çevirme)
ad_buyuk = musteri_verisi["ad_soyad"].upper()

# Random Kütüphanesi ve String Formatlama (f-string)
customerID = f"IST-2026-{{random.randint(1000, 9999)}}}"

# --- 2. KISIM: İLERİ SEVİYE YAPI ---

# Liste İçinde Sözlük Yapısı (Karmaşık Veri Yapıları)
musteriler_listesi = [
    {"ad": "Ahmet", "ucret": 550, "sure": 30},
    {"ad": "Ayşe", "ucret": 420, "sure": 12},
    {"ad": "Mehmet", "ucret": 600, "sure": 45}
]

# Fonksiyon Tanımlama (Parametre ve Return Kullanımı)
def tutar_hesapla(ucret):
    return ucret * 1.20 # %20 KDV ekleyip geri döndürme

# math Kütüphanesi Kullanımı (Yukarı Yuvarlama)
toplam_fatura = math.ceil(tutar_hesapla(musteri_verisi["aylik_ucret"]))

# datetime Kütüphanesi Kullanımı (Tarih Formatlama)
tarih = datetime.datetime.now().strftime("%d-%m-%Y")

```

```

# Set (Küme) Kullanımı (Benzersiz Eleman Bulma / Veri Temizleme)
tekrarlı_hizmetler = ["İnternet", "TV", "İnternet", "Mobil"]
benzersiz_hizmetler = set(tekrarlı_hizmetler)

# Çıktı Alma (Ekran Yazdırma)
print(f"ID: {{customerID}} | Müşteri: {{ad_buyuk}} | Durum: {{status}}")
print(f"Fatura Tutarı (KDV Dahil): {{toplam_fatura}} TL | Tarih: {{tarih}}")
print(f"Hizmet Çeşitleri: {{benzersiz_hizmetler}}")

```

Teslim Edilecekler:

- Fonksiyonları ve döngüleri içeren tam Python kodu.
- Kritik Soru Cevabı:** "Yazdığınız döngü (for loop) müşteri listesini gezerken, hangi müşterilerin Churn (ayrılma) riskinde olduğunu nasıl tespit ettiniz? Mantığınızı açıklayın."

Müşteri kaybını (Churn) tespit etmek için döngü içerisinde "**Negatif Göstergeler**" üzerinden bir mantık kurdum. Bir müşterinin sistemden ayrılma riskini belirleyen iki temel kriteri baz aldım:

1. **Aktiflik Durumu (aktif_mi == False):** Eğer bir müşterinin kaydı sistemde "Pasif" olarak işaretlenmişse, bu en net Churn belirtisidir.
2. **Düşük Sadakat ve Yüksek Ücret Dengesi:** Eğer bir müşteri çok yeniyle (sadakat ayı düşükse) ve aynı zamanda ödediği ücret ortalamanın üzerindeyse, bu müşteri rakiplerin tekliflerine daha açık olabilir.

Döngüdeki Uygulama Mantığı: Döngü (for loop) ile her bir müşteri sözlüğüne tek tek eriştik. Her adımda if kontrolü yaparak;

- Müşterinin aktif_mi değeri False mu?
- Veya sadakat_ayı belirli bir eşik değerinin (örneğin 3 ay) altında mı?

gibi sorular sorduk. Bu şartları sağlayan müşterileri döngü esnasında ayıklayarak "**Riskli Müşteri**" olarak etiketledik.

```
1 # Churn Risk Analizi Döngüsü (For Loop)
2
3 print("--- Müşteri Risk Analiz Raporu ---")
4
5 for musteriler in musteriler_listesi: # Döngü Kullanımı: Listeyi geziyoruz
6     # Mantık: Eğer müşteri aktif değilse VEYA sadakat süresi 6 aydan azsa risklidir
7     if not musteriler["aktif_mi"] or musteriler["sadakat_ay"] < 6:
8         durum = "RİSKLİ (Churn Olasılığı Yüksek)"
9     else:
10        durum = "Güvenli (Sadık Müşteri)"
11
12    # Her satırın hangi konuya ait olduğunu belirtiyoruz
13    print(f"Müşteri: {musteriler['ad_soyad']} | Durum: {durum}") # String Formatlama
```

--- Müşteri Risk Analiz Raporu ---

Müşteri: Fatih Eroğlu | Durum: Güvenli (Sadık Müşteri)

Müşteri: Simge Polat | Durum: Güvenli (Sadık Müşteri)

Müşteri: Cüneyt Çakır | Durum: RİSKLİ (Churn Olasılığı Yüksek)

Müşteri: Seren Serengil | Durum: RİSKLİ (Churn Olasılığı Yüksek)

Müşteri: Selahattin Çakaler | Durum: Güvenli (Sadık Müşteri)

